

CHAPTER 02

02. Node.js 내장 모듈 — 브라우저 저가 못 하던 일

—

Node.js · Express · NestJS 강의 · 2026

브라우저 안의 자바스크립트는 할 수 없는 일이 많다. 디스크의 파일을 읽거나, 포트를 열어 서버를 띄우거나, 환경변수를 들여다보는 일은 브라우저가 보안상 막아 둔 영역이다. Node.js는 자바스크립트를 브라우저 밖으로 꺼내 이런 일을 가능하게 한 런타임이고, 그 능력의 상당 부분이 **내장 모듈(built-in module)**로 들어 있다. `fs`, `path`, `http`, `events`, `os` 같은 이름들이다. 따로 설치할 것이 없다. Node를 깔면 함께 따라온다.

이 장에서는 이 내장 모듈들을 하나씩 직접 실행해 본다. 파일을 읽고 쓰고(`fs`), 경로를 다루고(`path`), 이벤트를 발생시키고(`events`), `http` 모듈만으로 웹 서버를 띄운다. 그에 앞서 **모듈을 가져오고 내보내는 두 가지 문법**(CommonJS와 ES Modules)부터 짚는다. 4장에서 만질 Express도 결국 이 `http` 모듈을 바탕으로 동작하고, `require`와 `import`는 앞으로 모든 장에서 매 줄 쓰게 된다. 이 장은 그 바닥이다.

이 자료는 **개념 예습용**이다. 실제 수업에서 쓰는 파일 이름과 폴더 구성, 다루는 범위는 강사에 따라 다를 수 있다. 그래서 여기서는 파일이 아니라 **무엇을 배우는가**에 초점을 둔다.

이 장에서 배우는 것

- 모듈 시스템 두 가지: **CommonJS**(`require` / `module.exports`)와 **ES Modules**(`import` / `export`), 그리고 그 둘을 가르는 `"type"` 설정
- 파일 다루기 `fs` — 동기·비동기·Promise 세 가지 방식과 그 차이
- 경로 다루기 `path` — OS마다 다른 구분자를 신경 쓰지 않고 경로를 조립하는 법
- 이벤트 `events` — `EventEmitter` 로 "사건이 나면 등록된 함수가 실행되는" 구조
- `http` 모듈만으로 서버 띄우기 — Express가 가려 두던 맨몸의 서버
- 그 밖에 `buffer` (바이너리), `stream` (조각 처리), `process` · `os` (실행 환경)

내용은 주제별로 쌓인다. 먼저 모듈 문법 두 가지를 잡고, 그다음 모듈을 하나씩 다룬다. 각 예제는 독립 실행되며 `node <파일명>` 으로 하나씩 돌린다.

주제	핵심
모듈 시스템	CommonJS(<code>require</code> / <code>module.exports</code>)와 ES Modules(<code>import</code> / <code>export</code>)
<code>fs</code>	파일 읽기·쓰기 — 동기·비동기·Promise
<code>path</code>	경로 조립·분해와 디렉터리 조작
<code>events</code>	<code>EventEmitter</code> — 이벤트 등록과 발생
<code>http</code>	<code>http</code> 모듈로 만든 웹 서버
<code>buffer</code>	<code>Buffer</code> — 바이너리·인코딩·Base64
<code>stream</code>	큰 데이터를 조각으로 처리
<code>process</code> · <code>os</code>	실행 환경 — 프로세스와 시스템 정보

핵심 개념 — 모듈은 "파일 하나가 곧 하나의 단위"다

자바스크립트 파일 한 개가 곧 하나의 **모듈**이다. 한 파일에서 만든 함수나 객체를 다른 파일에서 쓰려면, 내보내는(`export`) 쪽과 가져오는(`import`) 쪽의 약속이 필요하다. Node에는 이 약속이 두 가지 있다.

- **CommonJS (CJS)** — Node가 처음부터 써 온 방식. 내보낼 때 `module.exports`, 가져올 때 `require()`. 파일 확장자가 `.js` 이고 가까운 `package.json` 에 `"type": "commonjs"` (또는 `type` 자체가 없음)이면 이 방식으로 해석된다.
- **ES Modules (ESM)** — 표준 자바스크립트의 방식이고 브라우저에서도 쓰는 그 문법. 내보낼 때 `export`, 가져올 때 `import`. 확장자가 `.mjs` 이면 무조건 ESM이고, `.js` 라도 `package.json` 에 `"type": "module"` 이 있으면 ESM이다.

"type" 한 줄이 해석 방식을 바꾼다 — 같은 `.js` 파일이라도 `package.json` 의 `"type"` 값에 따라 CJS로도 ESM으로도 읽힌다. 이 장의 예제 폴더에는 `"type": "commonjs"` 가 박혀 있는데, 저장소 루트의 `package.json` 이 `"type": "module"` 이라 그것을 덮어쓰기 위함이다. 즉 이 폴더 안의 `.js` 는 전부 CommonJS로 동작하고, ESM을 보려면 확장자를 `.mjs` 로 둔 파일을 쓴다. ESM 예제만 확장자가 `.mjs` 인 이 유가 이것이다.

내장 모듈 쪽도 이름만 알면 된다. `require("fs")`, `require("http")` 처럼 경로가 아니라 **이름**을 적으면 Node가 자기 안에 들고 있는 모듈을 찾는다. 내가 만든 파일은 `require("./modules/math")` 처럼 `./` 로 시작하는 **경로**로 가져온다. 이 점 — "이름이면 내장 모듈, 경로면 내 파일" — 이 둘을 구분하는 기준이다.

준비

설치할 것이 없다. 이 장은 외부 패키지를 하나도 쓰지 않으므로 `npm install` 이 필요 없다. 예제 파일은 `node <파일명>` 으로 하나씩 실행한다.

```
node <파일명> # 예제 파일을 하나씩 실행
```

웹 서버 예제만 서버를 띄워 그 자리에서 멈춘 듯 보인다. 멈춘 게 아니라 요청을 기다리는 중이고, 종료는 `Ctrl+C` 다. 나머지 예제는 출력을 마치면 알아서 끝난다. 예제 폴더의 `package.json` 에 `"type": "commonjs"` 한 줄만 있는 이유는 위 핵심 개념에서 설명한 그대로다.

모듈 시스템 — `require` VS `import`

CommonJS — `module.exports` 와 `require`

```
const math = require("./modules/math");
const { add, subtract } = require("./modules/math");
const user = require("./modules/user");

console.log(=== CommonJS 모듈 시스템===);
console.log("math.add(5, 3):", math.add(5, 3));
console.log("math.subtract(10, 4):", math.subtract(10, 4));
console.log("구조 분해 할당으로 가져온 add(7, 2):", add(7, 2));
console.log("user 정보:", user);
```

`require("./modules/math")` 는 같은 폴더의 `math.js` 를 읽어 그 파일이 내보낸 값을 돌려준다. 가져오는 방식이 두 줄로 나뉘어 있는데, 둘은 같은 모듈을 다르게 받은 것이다.

- `const math = require(.)` — 모듈이 내보낸 객체 전체를 `math` 에 담는다. 이후 `math.add(.)` 처럼 점으로 꺼내 쓴다.
- `const { add, subtract } = require(.)` — 그 객체에서 `add`, `subtract` 만 구조 분해로 바로 꺼낸다. `math.add` 라고 매번 쓸 필요 없이 `add(7, 2)` 로 쓴다.

둘 중 무엇을 쓰든 동작은 같다. 자주 쓰는 함수 몇 개만 필요하면 구조 분해가 깔끔하고, 모듈 전체를 다룰 거면 통째로 받는다.

내보내는 쪽을 보자. `add` 와 `subtract` 를 가진 수학 모듈이다.

```
function add(a, b) {
  return a + b;
}

function subtract(a, b) {
  return a - b;
}

module.exports = {
  add,
  subtract,
};
```

`module.exports` 가 핵심이다. 이 변수에 담은 값이 곧 `require` 가 돌려주는 값이다. 여기서는 `add` 와 `subtract` 를 담은 객체를 통째로 내보냈다. 그래서 받는 쪽에서 `math.add` 로 꺼내거나 `{ add }` 로 분해할 수 있었다.

객체 하나를 그대로 내보내는 경우도 있다.

```
const user = {
  id: 1,
  name: "홍길동",
  email: "hong@example.com",
  role: "admin",
};

module.exports = user;
```

`module.exports` 에 함수 묶음이 아니라 객체 자체를 넣었다. 그래서 받는 쪽에서 `const user = require("./modules/user")` 로 받으면 `user` 가 바로 그 객체다. 내보낸 그 값이 받는 쪽에 그대로 온다 — 이것이 `module.exports` 의 전부다.

`module.exports` 와 `exports` — Node 모듈에는 `exports` 라는 단축 이름도 있는데, 처음엔 `exports` 가 `module.exports` 를 가리킨다. `exports.add = ..` 처럼 속성을 붙이는 건 괜찮지만, `exports = .{. }` 로 통째로 갈아끼우면 연결이 끊겨 아무것도 안 나간다. 헛갈리면 항상 `module.exports` 를 쓰면 된다.

ES Modules — `export` 와 `import`

```
import { sum, multiply } from "./modules/math.mjs";
import { formatDate } from "./modules/date.mjs";

console.log(ES 모듈 시스템);
console.log("sum(1, 2, 3, 4, 5):", sum(1, 2, 3, 4, 5));
console.log("multiply(2, 3, 4):", multiply(2, 3, 4));
console.log("formatDate(new Date()):", formatDate(new Date()));
```

같은 일을 ESM 문법으로 했다. `require` 자리에 `import` 가, `module.exports` 자리에 `export` 가 온다. 한 가지 눈여겨볼 차이가 있다. **ESM은 import 경로에 확장자(`.mjs`)를 붙여야 한다.** CommonJS의 `require("./modules/math")` 는 확장자를 생략해도 Node가 알아서 `.js` 를 붙여 찾지만, ESM에서는 `./modules/math.mjs` 처럼 끝까지 적는다.

내보내는 쪽을 보자.

```
export function sum(. numbers) {
  return numbers.reduce((acc, curr) => acc + curr, 0);
}

export function multiply(. numbers) {
  return numbers.reduce((acc, curr) => acc * curr, 1);
}
```

`module.exports = {.` } 로 한 번에 묶어 내보내던 CJS와 달리, ESM은 함수 선언 앞에 `export` 를 직접 붙인다. 이렇게 이름을 달아 내보낸 것을 **named export**라 하고, 받는 쪽은 `import { sum, multiply }` 처럼 그 이름 그대로 종괄호로 받는다. 이름이 맞아야 가져와진다.

ESM에서 날짜를 다루는 작은 예도 있다.

```
export function formatDate(date) {
  return new Intl.DateTimeFormat("ko-KR", {
    year: "numeric",
    month: "long",
    day: "numeric",
    hour: "2-digit",
    minute: "2-digit",
    second: "2-digit",
  }).format(date);
}
```

`Intl.DateTimeFormat` 은 Node와 브라우저 양쪽에 내장된 국제화 기능으로, `"ko-KR"` 로케를 주면 "2026년 6월 3일 오후 02:30:00" 같은 한국어 형식으로 날짜를 찍는다. 여기서 중요한 건 날짜 포맷이 아니라, 이 함수를 `export` 로 보내고 다른 파일에서 `import` 로 가져오는 흐름이다.

	CommonJS	ES Modules
가져오기	<code>require("./math")</code>	<code>import { sum } from "./math.mjs"</code>
내보내기	<code>module.exports = {.</code> }	<code>export function..</code>
확장자	생략 가능	명시 필수(<code>.mjs</code>)
적용 조건	<code>.js</code> + <code>type: commonjs</code>	<code>.mjs</code> 또는 <code>type: module</code>

모듈 캐싱 — 한 번 실행하고 결과를 공유한다

예제 폴더에는 위 예제가 직접 가져다 쓰진 않지만 개념 설명용 카운터 모듈도 있다.

```
let count = 0;

function getCount() {
  return count;
}

function increment() {
  count++;
}

module.exports = {
  getCount,
  increment,
};
```

이 카운터 모듈로 **모듈 캐싱**을 확인한다. Node는 한 모듈을 처음 `require` 할 때 한 번만 실행하고, 그 결과를 캐시에 저장해 둔다. 그래서 여러 파일에서 같은 카운터를 `require` 해도 `count` 변수는 **하나로 공유된다**. A 파일에서 `increment()` 를 부른 뒤 B 파일에서 `getCount()` 를 부르면 `1` 이 나온다. 모듈은 가져올 때마다 새로 만들어지지 않는다. 그래서 설정값이나 DB 연결처럼 "앱 전체가 하나만 공유해야 하는 것"을 모듈로 두는 패턴을 흔히 쓴다.

여러 내보내기 방식을 한 파일에 모아 둔 참고용 예제도 있다. `module.exports = defaultExport` 로 기본값을 깬 뒤 `module.exports.namedExport = ...` 로 속성을 더하고 `Object.assign` 으로 객체를 합치는 식인데, 실전에서는 이렇게까지 섞어 쓸 일이 드물다. "이런 것도 된다" 정도로 훑고 넘어가면 된다.

파일 다루기 — `fs`

`fs` (file system)는 디스크의 파일을 읽고 쓰는 내장 모듈이다. 여기서 다룰 주제는 **같은 작업을 세 가지 방식으로 한다는 것**이다. 동기, 비동기 콜백, Promise. 셋의 차이를 아는 게 이 절의 목표다.

```
const fs = require("fs");
const fsPromises = require("fs").promises;
```

`fs` 는 콜백·동기 함수들이 들어 있는 모듈이고, `fs.promises` 는 같은 기능을 Promise로 돌려주는 갈래다. 두 줄로 둘 다 손에 쥐었다.

동기 방식 — ~Sync

```
try {
  fs.writeFileSync("test.txt", "Hello World!");
  console.log("파일 쓰기 완료");

  const data = fs.readFileSync("test.txt", "utf8");
  console.log("파일 내용:", data);

  const stats = fs.statSync("test.txt");
  console.log("파일 크기:", stats.size);
  console.log("생성 시간:", stats.birthtime);
} catch (error) {
  console.error("에러:", error);
}
```

이름이 Sync 로 끝나는 함수는 동기다. 작업이 끝나야 다음 줄이 실행된다. `writeFileSync` 가 파일을 다 쓸 때까지 멈춰다가, 끝나면 `readFileSync` 로 넘어간다. 읽을 때 "utf8" 을 주면 글자(문자열)로, 안 주면 바이트 덩어리(Buffer)로 돌려준다. `statSync` 는 파일의 메타데이터 — 크기(`size`), 생성 시각(`birthtime`) 등 — 를 준다.

동기 함수는 에러를 throw 하므로 try/catch 로 감싼다. 코드가 위에서 아래로 곧장 읽혀 이해하기 쉽지만, 기다리는 동안 다른 일을 전혀 못 한다는 약점이 있다. 그래서 서버 코드에서는 잘 쓰지 않는다.

비동기 콜백 방식

```
fs.writeFile("async-test.txt", "Async Hello World!", (err) => {
  if (err) {
    console.error("쓰기 에러:", err);
    return;
  }
  console.log("비동기 파일 쓰기 완료");

  fs.readFile("async-test.txt", "utf8", (err, data) => {
    if (err) {
      console.error("읽기 에러:", err);
      return;
    }
    console.log("비동기 파일 내용:", data);
  });
});
```

Sync 가 없는 `writeFile` 은 비동기다. 파일 쓰기를 시작만 시켜 놓고 곧바로 다음 줄로 넘어가며, 쓰기가 끝나면 마지막 인자로 넘긴 콜백 함수를 실행한다. 콜백의 첫 인자는 항상 `err` 이다 — 에러가 없으면 `null` , 있으면 에러 객체가 들어온다. 이 "첫 인자가 에러"라는 규칙을 **에러 우선 콜백(error-first callback)** 이라 하고 Node 콜백의 표준 모양이다.

쓰기가 끝난 뒤에야 읽어야 하므로, `readFile` 을 `writeFile` 의 콜백 안에 넣었다. 작업이 줄줄이 이어지면 콜백이 콜백을 물고 계단처럼 깊어지는데, 이것을 흔히 콜백 지옥이라 부른다. 그 불편을 푼 것이 다음 Promise 방식이다.

Promise 방식 — `async` / `await`

```
async function fileOperations() {
  try {
    await fsPromises.writeFile("promise-test.txt", "Promise Hello World!");
    console.log("Promise 파일 쓰기 완료");

    const data = await fsPromises.readFile("promise-test.txt", "utf8");
    console.log("Promise 파일 내용:", data);

    const stats = await fsPromises.stat("promise-test.txt");
    console.log("Promise 파일 크기:", stats.size);
  } catch (error) {
    console.error("Promise 에러:", error);
  }
}

fileOperations();
```

`fsPromises`의 함수들은 콜백 대신 **Promise**를 돌려준다. 그 앞에 `await`를 붙이면 "이 작업이 끝날 때까지 기다렸다가 결과를 받아라"가 된다. `await`를 쓰는 함수는 앞에 `async`를 붙여야 한다.

겉보기엔 동기 코드처럼 위에서 아래로 곧게 읽히지만, 동작은 비동기다. `await`로 멈춰 있는 동안 Node는 다른 일을 처리할 수 있다. 동기의 가독성과 비동기의 효율을 둘 다 가진 셈이라, 요즘 Node 코드는 대부분 이 방식을 쓴다. 에러는 `try/catch`로 잡는다 — 동기와 같은 모양이다.

비동기는 정말 "기다리는 동안 논다"

마지막 블록이 비동기의 핵심을 눈으로 보여 준다.

```

async function concurrentOperations() {
  console.log("1. 작업 시작");

  const writePromise = fsPromises.writeFile(
    "concurrent-test.txt",
    "Concurrent Hello World!"
  );

  console.log("2. 파일 쓰기 작업 시작됨");

  console.log("3. 다른 작업 수행 중.");
  for (let i = 0; i < 3; i++) {
    console.log(`  다른 작업 ${i + 1} 실행 중`);
    await new Promise((resolve) => setTimeout(resolve, 100));
  }

  await writePromise;
  console.log("4. 파일 쓰기 완료");

  const data = await fsPromises.readFile("concurrent-test.txt", "utf8");
  console.log("5. 파일 내용:", data);
}

```

요령은 `writeFile`의 결과에 `await`를 바로 붙이지 않은 것이다. `const writePromise = fsPromises.writeFile(. )`로 쓰기를 시작만 시켜 `Promise`를 손에 쥐어 두고, 그 쓰기가 디스크에서 진행되는 동안 아래 `for` 루프(다른 작업)를 돌린다. 다른 작업을 마친 뒤 `await writePromise`로 그제야 쓰기 완료를 확인한다. 시작과 대기를 떼어 놓으면, 기다리는 시간 동안 다른 일을 겹쳐 할 수 있다 — 이것이 동기 방식으로는 안 되는, 비동기를 쓰는 이유다.

여러 줄의 출력 순서가 섞여 보일 수 있다 — 이 예제는 동기·비동기·`Promise` 블록을 한 번에 실행한다. 동기 출력이 먼저 다 찍히고, 비동기·`Promise` 결과는 작업이 끝나는 대로 나중에 찍히므로 화면의 줄 순서가 코드 순서와 다를 수 있다. 버그가 아니라 비동기의 정상 동작이다.

경로 다루기와 디렉터리 조작 — `path`

`path`는 파일 경로 문자열을 안전하게 조립·분해하는 모듈이다. 직접 `"폴더" + "/" + "파일"`로 이어 붙이면 안 되는데, 경로 구분자가 OS마다 다르기 때문이다(맥·리눅스는 `/`, 윈도우는 `\`). `path`는 그 차이를 알아서 맞춘다.

```

const fullPath = path.join(__dirname, "files", "test.txt");
console.log("전체 경로:", fullPath);

const pathParts = path.parse(fullPath);
console.log("경로 분리:", pathParts);

const ext = path.extname("test.txt");
console.log("확장자:", ext);

const absolutePath = path.resolve("test.txt");
console.log("절대 경로:", absolutePath);

const normalizedPath = path.normalize("/foo/bar//baz/asdf/quux/..");
console.log("정규화된 경로:", normalizedPath);

const relativePath = path.relative("/foo/bar", "/foo/bar/baz");
console.log("상대 경로:", relativePath);

```

핵심 함수들을 한자리에서 본다.

- `path.join(__dirname, "files", "test.txt")` — 조각들을 OS에 맞는 구분자로 이어 붙인다. `__dirname` 은 현재 파일이 있는 폴더의 절대 경로를 담은 변수로, CommonJS가 자동으로 채워 준다. 경로를 만들 때 이 기준점에서 출발하는 게 안전하다.
- `path.parse(.)` — 경로를 `root · dir · base · name · ext` 조각으로 쪼갠 객체를 준다. `path.join` 의 반대다.
- `path.extname("test.txt")` — 확장자만 떼어 `".txt"` 를 준다.
- `path.resolve("test.txt")` — 상대 경로를 현재 작업 디렉터리 기준의 절대 경로로 바꾼다.
- `path.normalize(.)` — 경로 안의 `//` (중복 구분자)나 `..` (상위로) 같은 군더더기를 정리한다. 예의 `quux/..` 는 서로 상쇄돼 사라진다.
- `path.relative("/foo/bar", "/foo/bar/baz")` — 앞 경로에서 뒤 경로로 가는 상대 경로(`baz`)를 계산한다.

디렉터리 만들고·읽고·지우기 — `fs`

같은 예제의 뒷부분은 `fs` 로 폴더를 다룬다. `path` 로 경로를 만들고 `fs` 로 실제 작업을 하는, 둘이 짝을 이루는 전형적인 모습이다.

```

const dirPath = path.join(__dirname, "new-directory");
if (!fs.existsSync(dirPath)) {
  fs.mkdirSync(dirPath);
  console.log("디렉토리 생성됨:", dirPath);
}

const files = fs.readdirSync(__dirname);
console.log("현재 디렉토리의 파일들:", files);

```

- `fs.existsSync(dirPath)` — 경로가 존재하는지 묻는다. 이미 있는 폴더를 또 만들려다 에러 나는 걸 막으려고 `if (!.)` 로 먼저 확인했다.
- `fs.mkdirSync(dirPath)` — 폴더를 만든다.
- `fs.readdirSync(__dirname)` — 폴더 안의 파일·폴더 이름을 배열로 준다.

조금 더 나아간 조작들도 이어진다.

```
const nestedDirPath = path.join(__dirname, "parent", "child", "grandchild");
fs.mkdirSync(nestedDirPath, { recursive: true });
console.log("중첩된 디렉토리 생성됨:", nestedDirPath);
```

`{ recursive: true }` 옵션이 요점이다. `parent` 도 `child` 도 없는데 `parent/child/grandchild` 를 한 번에 만들라고 하면 보통은 에러가 난다. `recursive` 를 켜면 중간 폴더까지 한꺼번에 만든다. `mkdir -p` 와 같은 동작이다.

이후 코드는 이름 변경(`renameSync`), 심볼릭 링크 생성(`symlinkSync`), 권한 변경(`chmodSync`), 소유자 변경(`chownSync`)으로 이어진다. `chownSync` 는 권한이 있어야 성공하므로 실패할 수 있고, 그래서 `try/catch` 로 감싸 에러 메시지만 찍고 넘어가게 해 뒀다. 이런 폴더 조작은 종류가 많지만, 결국 **존재 확인** → **생성** → **읽기** → **정리** 라는 흐름의 변주다.

이벤트 다루기 — EventEmitter

Node의 많은 객체가 "무슨 일이 일어나면 등록해 둔 함수를 부른다"는 방식으로 동작한다. 서버에 요청이 오면, 스트림에 데이터가 들어오면, 파일을 다 읽으면 — 모두 **이벤트**다. 그 바탕이 `events` 모듈의 `EventEmitter` 다.

```
const EventEmitter = require("events");

const emitter = new EventEmitter();

emitter.on("greet", (name) => {
  console.log(`안녕하세요, ${name}님!`);
});

emitter.emit("greet", "홍길동"); // "안녕하세요, 홍길동님!"
emitter.emit("greet", "김철수"); // "안녕하세요, 김철수님!"
```

구조는 단 두 동작으로 끝난다.

- `emitter.on("greet", 함수)` — `"greet"` 라는 이름의 이벤트가 발생하면 이 함수를 부르라고 **등록(구독)** 한다. 등록된 함수를 **리스너(listener)** 라 한다.
- `emitter.emit("greet", "홍길동")` — `"greet"` 이벤트를 **발생**시킨다. `emit` 의 두 번째 인자부터는 리스너에 그대로 전달된다. 여기서는 `"홍길동"` 이 리스너의 `name` 으로 들어간다.

"발생시키는 쪽"과 "처리하는 쪽"이 이벤트 이름으로만 느슨하게 연결되는 게 핵심이다. `emit` 하는 코드는 누가 듣는지 몰라도 되고, `on` 하는 코드는 언제 발생할지 몰라도 된다.

하나의 이벤트, 여러 리스너

```
orderEmitter.on("order", (item) => console.log(` [재고팀] ${item} 출고 준비`));
orderEmitter.on("order", (item) => console.log(` [배송팀] ${item} 배송 라벨 생성`));
orderEmitter.on("order", (item) => console.log(` [알림팀] ${item} 주문 확인 메일 발송`));

orderEmitter.emit("order", "노트북");
```

같은 `"order"` 이벤트에 리스너를 셋 등록했다. `emit` 한 번에 셋이 등록된 순서대로 모두 실행된다. 주문 하나가 들어오면 재고·배송·알림이 각자 반응하는 구조를, 서로 모르는 채로 깔끔하게 나눠 둘 수 있다.

once 와 리스너 제거

```
onceEmitter.once("welcome", () => console.log(" 최초 1회만 실행됨"));
onceEmitter.emit("welcome"); // 실행
onceEmitter.emit("welcome"); // 무시
```

`once` 로 등록한 리스너는 딱 한 번 실행되고 스스로 떨어져 나간다. 초기화처럼 한 번만 해야 하는 일에 쓴다.

```
const handler = () => console.log(" 리스너 동작 중");
removable.on("tick", handler);
removable.emit("tick"); // 동작
removable.off("tick", handler);
removable.emit("tick"); // 동작 안 함
```

`off(이름, 함수)` 로 등록을 떼어 낸다. 떼어 내려면 등록할 때 쓴 함수를 그대로 넘겨야 하므로, 익명 함수가 아니라 `handler` 라는 이름에 담아 둔 것이다.

클래스 상속 — 내 객체를 이벤트 발행자로

```
class ChatRoom extends EventEmitter {
  constructor(name) {
    super();
    this.name = name;
    this.users = [];
  }

  join(user) {
    this.users.push(user);
    this.emit("join", user);
  }

  send(user, message) {
    this.emit("message", { user, message, time: new Date() });
  }

  leave(user) {
    this.users = this.users.filter((u) => u !== user);
    this.emit("leave", user);
  }
}
```

`extends EventEmitter` 로 상속하면 내가 만든 클래스가 곧 이벤트 발행자가 된다. `ChatRoom` 은 입장·메시지·퇴장이라는 사건을 `this.emit(.)` 으로 내보내고, 바깥에서는 그 사건을 `room.on(.)` 으로 구독한다.

```
const room = new ChatRoom("개발자방");

room.on("join", (user) => console.log(` >> ${user}님이 입장했습니다`));
room.on("message", ({ user, message }) => console.log(` ${user}: ${message}`));
room.on("leave", (user) => console.log(` >> ${user}님이 퇴장했습니다`));

room.join("홍길동");
room.send("홍길동", "안녕하세요!");
room.leave("홍길동");
```

`ChatRoom` 은 "입장하면 사건을 알린다"까지만 책임지고, 그 사건에 무엇을 할지(화면 출력, 로그, 알림)는 구독하는 쪽이 정한다. 기능을 추가하고 싶으면 클래스를 고치는 게 아니라 `on` 을 하나 더 붙이면 된다. 이 느슨한 분리가 이벤트 방식의 가장 큰 장점이다.

error 이벤트와 리스너 개수

```
errorEmitter.on("error", (err) => {
  console.log(" 에러 잡힘:", err.message);
});

errorEmitter.emit("error", new Error("뭔가 잘못됨"));
```

"error" 는 EventEmitter 에서 특별 취급되는 이름이다. "error" 이벤트를 emit 했는데 그걸 듣는 리스너가 하나도 없으면, Node는 에러를 던지며 프로세스를 통째로 종료한다. 그래서 emit("error", ...) 전에 반드시 on("error", ...) 로 받아 뒀야 한다.

```
console.log(" 기본 최대치:", limited.getMaxListeners()); // 10
limited.setMaxListeners(20);
```

한 이벤트에 리스너를 11개 이상 달면 Node가 "메모리 누수 아니냐"는 경고를 띄운다. 기본 한도가 10개라서다. 정말 많이 달아야 하면 setMaxListeners 로 한도를 올린다. 평소엔 신경 쓸 일이 없고, 경고를 만났을 때 떠올리면 되는 안전장치다.

http 모듈로 서버 만들기

이 절이 이 장의 정점이다. Express 없이, http 모듈만으로 웹 서버를 띄운다. 4장에서 배울 Express가 무엇을 대신하는지 알려면, 그 아래 맨몸의 서버를 먼저 봐야 한다.

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "text/plain; charset=utf-8" });

  console.log("요청 URL:", req.url);
  console.log("요청 메서드:", req.method);
  console.log("요청 헤더:", req.headers);

  res.end("안녕하세요! Node.js 웹 서버입니다.");
});

const PORT = 3000;
server.listen(PORT, () => {
  console.log(`서버가 http://localhost:${PORT} 에서 실행 중입니다.`);
});
```

전체가 이게 다다. 한 줄씩 보자.

- `http.createServer((req, res) => .{. })` — 서버를 만든다. 인자로 준 함수는 **요청이 들어올 때마다** 실행되는 핸들러다. `req` (요청)와 `res` (응답)를 받는데, 4장 Express의 `(req, res)` 와 같은 이름이다. Express의 그 두 인자가 사실 여기서 온 것이다.
- `res.writeHead(200, .{. })` — 응답의 **상태코드와 헤더**를 먼저 쓴다. `200` 은 성공, `Content-Type` 은 "내가 보내는 게 어떤 형식인지"를 알린다. 한글이 깨지지 않게 `charset=utf-8` 을 붙였다.
- `req.url` , `req.method` , `req.headers` — 들어온 요청의 경로·메서드·헤더다. 여기서는 콘솔에 찍기만 한다.
- `res.end(?. )` — 응답 본문을 실어 보내고 응답을 **끝낸다**. `res.end` 를 부르지 않으면 클라이언트는 응답이 안 끝난 줄 알고 계속 기다린다.
- `server.listen(PORT, 콜백)` — 3000번 포트에서 요청을 받기 시작한다. 이 줄이 있어야 서버가 켜진다. 콜백은 서버가 준비됐을 때 한 번 실행된다.

예제를 실행하면 차이가 분명해진다.

```
node <파일명>
# 서버가 http://localhost:3000 에서 실행 중입니다.
```

브라우저로 `http://localhost:3000` 에 접속하면, **어떤 경로로 들어가도** 똑같이 "안녕하세요! Node.js 웹 서버입니다."가 나온다. `/` 든 `/posts` 든 `/아무거나` 든 마찬가지다. 이 서버에는 경로 분기가 없기 때문이다. 종료는 `Ctrl+C` 다.

이 서버의 한계가 곧 Express의 존재 이유다 — `/posts` 와 `/users` 를 다르게 처리하려면 이 핸들러 안에서 `if (req.url === "/posts")` 같은 분기를 직접 짜야 한다. POST 본문(JSON)을 받으려면 `req` 에 들어오는 데이터 조각을 직접 모아 `JSON.parse` 까지 해야 한다. 라우트가 늘수록 이 `if` 더미가 감당이 안 된다. 4장의 Express 는 바로 이 반복을 `app.get("/posts", ...)` , `express.json()` 같은 짧은 형태로 걷어 낸 것이다. Express도 내부에서는 결국 이 `http.createServer` 를 쓴다. 우리가 지금 본 게 그 바닥이다.

요청 한 번의 흐름은 4장과 똑같다.

```
브라우저 — GET / —> http 서버 —> 핸들러(req, res) 실행 —> res.end(.) —> 브라우저
```

바이너리 데이터와 인코딩 — Buffer

문자열만으로는 다룰 수 없는 데이터가 있다. 이미지, 압축 파일, 네트워크로 오가는 바이트들이다. 자바스크립트의 문자열은 글자를 위한 것이지 **바이트(0~255)** 를 위한 것이 아니다. 그 빈자리를 메우는 게 `Buffer` 다.


```
const fromString = Buffer.from("Hello, Node!");
console.log("from string:", fromString); // <Buffer 48 65 6c 6c 6f...>
console.log("length:", fromString.length); // 12

const korean = Buffer.from("안녕");
console.log("한글 length:", korean.length); // 6 (글자 2개인데 6바이트)
```

- `Buffer.from("Hello, Node!")` — 문자열을 바이트 배열로 바꾼다. 찍어 보면 각 글자의 바이트 값이 16진수로 나온다(48 은 H).
- `korean.length` 가 6 인 게 포인트다. 한글 "안녕"은 글자로는 2개지만, UTF-8 인코딩에서 한 글자가 3바이트라 총 6바이트다. `Buffer` 는 글자 수가 아니라 **바이트** 수를 센다. 글자 수와 바이트 수가 다르다는 점은 파일 크기나 전송량을 계산할 때 자주 헷갈린다.

```
const empty = Buffer.alloc(10); // 0으로 채워진 10바이트
const uninit = Buffer.allocUnsafe(10); // 빠르지만 이전 메모리 잔재 가능
```

`alloc(10)` 은 0으로 채운 안전한 10바이트 버퍼다. `allocUnsafe(10)` 은 메모리를 초기화하지 않아 더 빠르지만, 이전에 그 메모리에 있던 값이 그대로 남아 있을 수 있다. 그 값을 곧바로 덮어쓸 게 아니라면 `alloc` 을 쓰는 게 안전하다.

인코딩 변환과 Base64

```
const buf = Buffer.from("안녕하세요");
console.log("UTF-8 String:", buf.toString("utf-8"));
console.log("Hex String:", buf.toString("hex"));
console.log("Base64 String:", buf.toString("base64"));
```

같은 바이트 덩어리를 `toString` 에 어떤 인코딩을 주느냐에 따라 다르게 풀어낸다. `utf-8` 이면 원래 글자로, `hex` 면 16진수 문자열로, `base64` 면 Base64 문자열로 나온다.

```
const original = "비밀번호:abc123";
const encoded = Buffer.from(original).toString("base64");
const decoded = Buffer.from(encoded, "base64").toString("utf-8");
```

Base64는 바이너리를 글자만으로 표현하는 인코딩이다. 이미지나 파일을 JSON·HTTP 헤더처럼 텍스트만 허용하는 자리에 실어 보낼 때 쓴다. 인코딩(`toString("base64")`)과 디코딩(`Buffer.from(encoded, "base64")`)이 서로 짝이라는 것만 잡으면 된다.

이어지는 `bytes[0]` (인덱스로 바이트 접근), `subarray` (부분 자르기), `concat` (이어 붙이기), `compare · equals` (비교)는 모두 바이트 덩어리를 배열처럼 다루는 기본 연산이다. 예제 끝의 "실전 시나리오"는 이미지 파일을 읽어 Base64로 바꿔 JSON 응답에 임베드하는 예를 코드만 보여 준다(실제 파일이 없어 출력은 안 한다). Base64가 어디에 쓰이는지 감만 잡으면 된다.

큰 데이터를 조각으로 — 스트림

1GB짜리 로그 파일을 `readFileSync` 로 읽으면 메모리에 1GB가 통째로 올라온다. 메모리가 감당 못 하면 프로세스가 죽는다. **스트림(stream)** 은 데이터를 한 번에 다 읽지 않고 작은 조각(chunk)으로 나눠 흘려보내며 처리한다. 비디오 스트리밍, 큰 파일 전송, 대용량 로그 처리가 모두 이 방식이다.

```
const testFile = path.join(__dirname, "stream-test.txt");
const lines = Array.from({ length: 10000 }, (_, i) => `라인 ${i + 1}: 샘플 텍스트입니
다.`).join("\n");
fs.writeFileSync(testFile, lines);
```

먼저 1만 줄짜리 테스트 파일을 만든다. 스트림으로 읽을 거리를 마련하는 준비다.

Readable Stream — 조각 단위로 읽기

```
const readStream = fs.createReadStream(testFile, { encoding: "utf-8" });

let chunkCount = 0;
let totalBytes = 0;

readStream.on("data", (chunk) => {
  chunkCount += 1;
  totalBytes += chunk.length;
  if (chunkCount === 1) {
    console.log(` 첫 chunk 미리보기: "${chunk.substring(0, 40)}". "`);
  }
});

readStream.on("end", () => {
  console.log(` 총 ${chunkCount}개 chunk, ${totalBytes}바이트 읽음\n`);
  //...
});
```

`fs.createReadStream` 이 읽기 스트림을 만든다. 여기서 앞서 본 `EventEmitter` 가 다시 등장한다. **스트림은 이벤트로 동작한다.**

- `on("data", chunk => ..)` — 조각이 하나 들어올 때마다 실행된다. 큰 파일이면 여러 번 호출된다. 기본 조각 크기는 64KB다.
- `on("end", ...)` — 더 읽을 게 없을 때 한 번 실행된다. 읽기 작업의 마무리를 여기서 한다.

파일을 통째로 메모리에 올리지 않고 조각이 들어오는 족족 처리하므로, 파일이 아무리 커도 메모리는 조각 하나 크기면 충분하다.

pipe — 읽기를 쓰기로 곧장 연결

```
const src = fs.createReadStream(testFile);
const dst = fs.createWriteStream(copyFile);

src.pipe(dst); // 한 줄로 파일 복사

dst.on("finish", () => {
  console.log(" 복사 완료:", copyFile);
  //...
  fs.unlinkSync(testFile);
  fs.unlinkSync(copyFile);
});
```

`pipe` 가 스트림의 꽃이다. 읽기 스트림(`src`)을 쓰기 스트림(`dst`)에 연결하면, 한쪽에서 나온 조각이 다른 쪽으로 자동으로 흘러 들어간다. 위 한 줄이 파일 복사다. 직접 `data` 이벤트를 받아 `write` 로 옮길 필요 없이, `pipe` 가 조각의 흐름과 속도 조절까지 알아서 한다. 쓰기가 끝나면 `finish` 이벤트가 발생하고, 거기서 `unlinkSync` 로 테스트 파일들을 지워 정리한다.

예제 끝의 두 "코드 예시"는 출력만 하고 실행은 안 한다. HTTP 응답으로 큰 파일을 스트리밍하는 예(`file.pipe(res)`)와, 데이터를 변환하며 흘러보내는 `Transform` 스트림 예다. 특히 `file.pipe(res)` 는 기억해 둘 만하다. 응답 객체 `res` 도 쓰기 스트림이라, 파일을 메모리에 다 올리지 않고 클라이언트로 바로 흘러보낼 수 있다. 큰 파일 다운로드를 구현하는 표준 방식이다.

실행 환경 들여다보기 — `process` 와 `os`

`process` 는 지금 돌아가는 Node 프로세스 자신에 대한 정보를, `os` 는 그 프로세스가 올라타 있는 운영체제에 대한 정보를 준다. `require` 없이도 쓸 수 있는 전역 객체 `process` 부터 보자.

```
console.log("Node.js 버전:", process.version);
console.log("플랫폼:", process.platform);
console.log("아키텍처:", process.arch);

console.log("NODE_ENV:", process.env.NODE_ENV);
console.log("PATH:", process.env.PATH);
```

- `process.version` / `process.platform` / `process.arch` — Node 버전, OS 종류 (`darwin` · `linux` · `win32`), CPU 아키텍처(`arm64` · `x64`)다.
- `process.env` — 환경변수가 담긴 객체다. `process.env.NODE_ENV` , `process.env.PATH` 처럼 꺼낸다. 환경변수는 코드 밖에서 설정값을 주입하는 표준 통로로, 3장에서 `dotenv` 로 본격적으로 다룬다.

```
process.on("SIGINT", () => {
  console.log("\n프로그램 종료");
  process.exit();
});
```

여기서도 이벤트가 등장한다. `process` 도 `EventEmitter` 라, `Ctrl+C` 를 누르면 OS가 보내는 `SIGINT` 신호(signal)를 이벤트로 받을 수 있다. 이 핸들러를 등록해 두면 종료 직전에 정리 작업(연결 닫기, 로그 남기기)을 끼워 넣을 수 있다. 04장에서 서버를 `Ctrl+C` 로 끝 때 내부적으로 일어나는 일이 바로 이것이다.

```
console.log("힙 사용량:", process.memoryUsage().heapUsed);
```

`process.memoryUsage()` 는 이 프로세스가 쓰는 메모리를 알려준다. 뒤이어 나오는 `os` 모듈은 프로세스 바깥, 시스템 전체를 본다.

```
const os = require("os");

console.log("운영체제:", os.type());
console.log("호스트명:", os.hostname());

const cpus = os.cpus();
console.log(`CPU 코어 수: ${cpus.length}`);

console.log(`총 메모리: ${(os.totalmem() / 1024 / 1024 / 1024).toFixed(2)} GB`);
console.log("시스템 부하 평균 (1, 5, 15분):", os.loadavg());
```

`os.cpus()` 로 코어 수와 모델, `os.totalmem() / os.freemem()` 으로 전체·여유 메모리, `os.networkInterfaces()` 로 IP, `os.uptime()` 으로 가동 시간, `os.loadavg()` 로 부하 평균을 얻는다. `process` 가 "나(프로세스)"를 본다면 `os` 는 "내가 사는 집(시스템)"을 본다 — 이 대비로 둘을 기억하면 된다.

이 예제는 그대로는 실행되지 않는다 — 한 예제에 `const os = require('os')` 와 `process.exit` 종료 블록이 각각 두 번씩 들어가 있다. 같은 이름을 두 번 `const` 로 선언할 수 없어 `SyntaxError: Identifier 'os' has already been declared` 가 난다. 두 예제(`process` 중심 / `os` 중심)를 하나로 이어 붙이며 생긴 중복이다. 직접 돌려 보려면 둘째 `const os = require('os');` 와 겹치는 종료 블록 한쪽을 지우면 된다. 흔한 실수에서 이런 중복 선언을 다시 다룬다.

정리 표 — 이 장의 내장 모듈

모듈	무엇을 하나	대표 함수·객체
(모듈 시스템)	파일 간 코드 공유	<code>require</code> / <code>module.exports</code> , <code>import</code> / <code>export</code>
<code>fs</code>	파일·디렉터리 읽기·쓰기	<code>readFile</code> · <code>writeFile</code> · <code>stat</code> · <code>mkdir</code> (+ <code>Sync</code> / <code>promises</code>)
<code>path</code>	경로 조립·분해	<code>join</code> · <code>parse</code> · <code>resolve</code> · <code>extname</code>
<code>events</code>	이벤트 발행·구독	<code>EventEmitter</code> , <code>on</code> · <code>emit</code> · <code>once</code> · <code>off</code>
<code>http</code>	웹 서버	<code>createServer</code> · <code>listen</code> , <code>req</code> · <code>res</code>
<code>buffer</code>	바이너리·인코딩	<code>Buffer.from</code> · <code>toString</code> · <code>alloc</code>
<code>stream</code>	큰 데이터 조각 처리	<code>createReadStream</code> · <code>pipe</code>
<code>process</code>	프로세스 정보·환경변수	<code>env</code> · <code>version</code> · <code>on("SIGINT")</code>
<code>os</code>	시스템 정보	<code>cpus</code> · <code>totalmem</code> · <code>hostname</code>

흔한 실수

- **CommonJS와 ES Modules를 섞는다.** `.js` 파일에서 `import` 를 쓰면(또는 `.mjs` 에서 `require` 를 쓰면) `type` 설정에 따라 에러가 난다. 이 장의 예제 폴더는 `"type": "commonjs"` 이므로 `.js` 는 `require`, ESM은 `.mjs` 파일로 분리돼 있다.
- **ESM에서 import 경로의 확장자를 빠뜨린다.** CommonJS의 `require("./math")` 는 되지만, ESM은 `import ... from "./math.mjs"` 처럼 `.mjs` 까지 써야 한다.
- **같은 이름을 const 로 두 번 선언한다.** 한 예제에서 `const os` 가 두 번 나와 `SyntaxError` 가 나는데, 한 파일에 예제를 이어 붙일 때 자주 생긴다.
- **읽기 전용 작업에 동기 함수(~Sync)를 남발한다.** `readFileSync` 는 일이 끝날 때까지 다른 요청을 다 막는다. 서버 코드에서는 비동기(`fs.promises` + `await`)를 쓴다.
- **res.end() 를 부르지 않는다.** `http` 서버에서 응답을 끝맺지 않으면 클라이언트가 영영 기다린다.
- **"error" 이벤트에 리스너를 안 단다.** `EventEmitter` 에서 `error` 를 듣는 곳이 없으면 프로세스가 통째로 죽는다.

직접 해보기

1. CommonJS 수학 모듈에 곱셈 함수 `multiply(a, b)` 를 추가하고, `module.exports` 에 넣어 메인 파일에서 `require` 로 가져와 출력해 보자. CommonJS의 `export/import` 흐름이 손에 익는다.

2. `http` 서버 핸들러에서 `req.url` 을 보고 경로마다 다른 응답을 돌려주게 고쳐 보자. `if (req.url === "/")` 면 환영 인사, `/time` 이면 `new Date()` 를 보내는 식이다. Express 없이 직접 라우팅을 짜 보면, 4장의 `app.get` 이 무엇을 대신하는지 체감된다.
3. 중복된 `const os` 와 종료 블록 한쪽을 지워 정상 실행되게 고친 뒤, `process` 정보만 찍는 파일과 `os` 정보만 찍는 파일로 둘로 나눠 보자.

정리

- 모듈은 파일 단위이고, 가져오고 내보내는 문법이 둘 있다: **CommonJS**(`require` / `module.exports`)와 **ES Modules**(`import` / `export`). `package.json` 의 `"type"` 이 해석 방식을 가른다.
- `fs` 는 같은 작업을 동기·콜백·Promise 세 방식으로 제공하며, 서버에서는 `fs.promises` + `await` 를 쓴다. 경로는 직접 잊지 말고 `path.join` 으로 조립한다.
- `EventEmitter` 의 `on` / `emit` 은 "사건이 나면 등록된 함수가 실행되는" 구조이고, 스트림·서버·`process` 가 모두 이 위에 서 있다.
- `http.createServer` 만으로도 서버는 뜨지만, 경로 분기와 본문 파싱을 전부 손으로 짜야 한다. 그 반복을 걷어 낸 게 다음에 만질 도구다.

다음 장(**03·외부 모듈**)에서는 내장 모듈을 넘어 `dotenv` · `winston` · `validator` · `dayjs` 같은 npm 패키지를 설치해 쓰며, `npm install` 과 `package.json` 이 실제로 어떻게 돌아가는지 본다.